

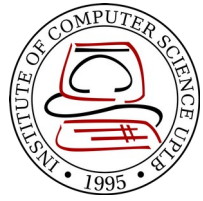
CMSC 137

Data Communications and Networking

Lecture Module
12 – Data Link Layer



University of the Philippines
LOS BAÑOS



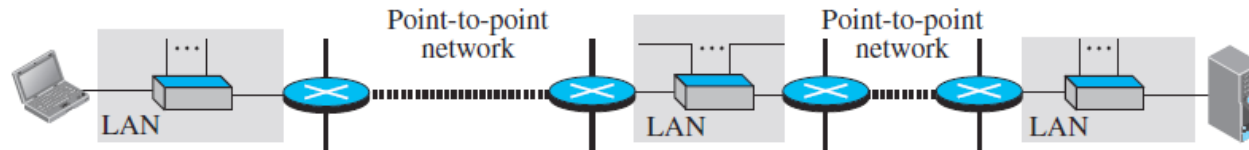
Module 12: Data Link Layer

Rozano S. Maniaol
Lecturer

Introduction



- TCP/IP Protocol Suite does not define any protocol in the data link or physical layer
- The data link layer transforms the physical layer to a link responsible for **node to node communication**
- It is customary to refer to end hosts and the routers as **nodes** and the network in between as **links**

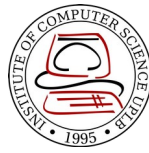


a. A small part of the Internet

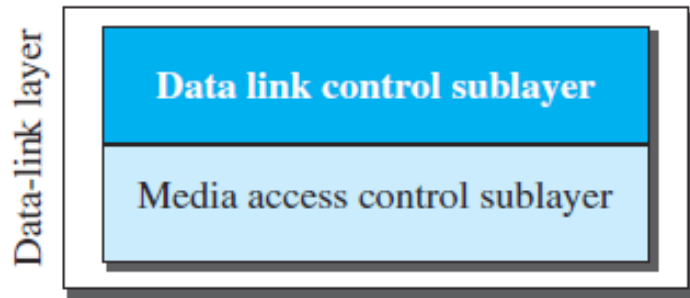


b. Nodes and links

Introduction



- Main functions are **data link control** and **media access control**
- Two Categories of Link
 - **Point-to-point link** – link is dedicated to the two devices
 - **Broadcast link** – link is shared between several pairs of devices
- Two Sublayers



a. Data-link layer of a broadcast link

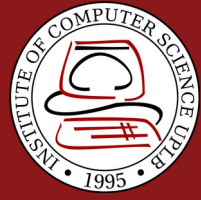


b. Data-link layer of a point-to-point link

Services

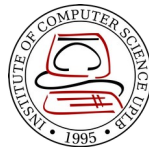


- Framing
- Flow Control
- Error Control
- Media Access Control
- Link Layer Addressing



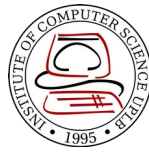
Link Layer Addressing

Link Layer Addressing



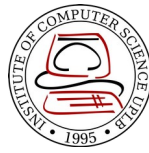
- In a connectionless internetwork such as the Internet, we cannot make a datagram reach its destination using only IP addresses
- Source and destination IP address define the 2 ends but cannot define which links the datagram should pass
- Another addressing mechanism needed in connectionless internetwork: [link layer address](#) / [physical address](#) / [MAC address](#)

Types of Addresses

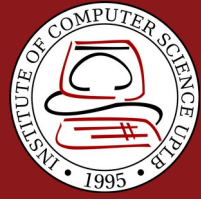


- **Unicast address** – used in one to one communication
 - In Ethernet, addresses are 48 bits (six bytes) presented as 12 hexadecimal digits separated by colons; a unicast address has an even number in the second digit in hexadecimal
 - Ex. A2:34:45:11:92:F1
- **Multicast address** – used in one-to-many communication but the jurisdiction is local (inside the link)
 - In Ethernet, the second digit is an odd number in hexadecimal
 - Ex. A3:34:45:11:92:F1

Types of Addresses

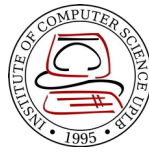


- **Broadcast address** – one-to-all communication; frame with destination broadcast address is sent to all entities in the link
 - In Ethernet, the 48 bits are all 1s that are presented as 12 hexadecimal digits separated by colons
 - Ex. FF:FF:FF:FF:FF:FF

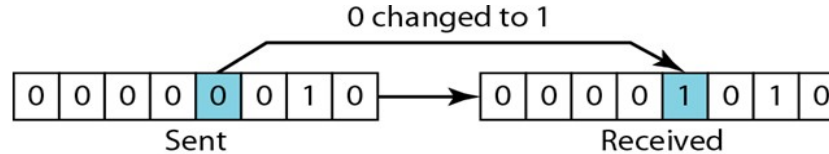


Error Correction and Detection

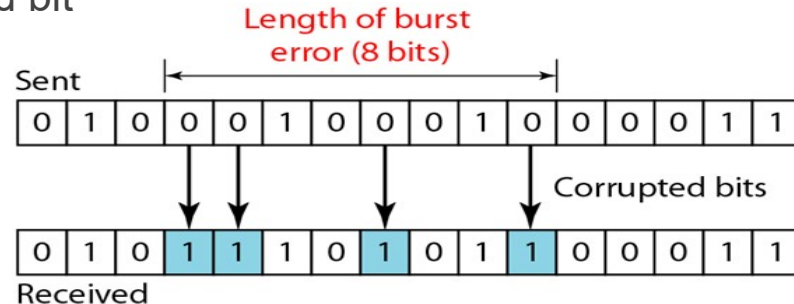
Error Detection and Correction



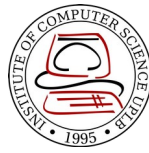
- Data can be corrupted during transmission because of interference
- Types of Errors
 - Single-bit errors** – only 1 bit is changed in the data unit



- Burst errors** – two or more bits have changed, length is measured from the first corrupted bit to the last corrupted bit
 - Affected bits depend on data rate and noise duration

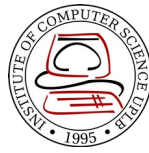


Error Detection and Correction



- Redundancy
 - To be able to detect or correct errors, extra bits are needed with data
 - Redundant bits are added by the sender and removed by the receiver
- Detection vs. Correction
 - Detection – did errors occur?
 - Correction – need to know the exact number of corrupted bits and exact location

Error Detection and Correction



- Forward Error Correction
 - Receiver tries to guess the message by using redundant bits
- Re-transmission
 - Receiver detects an error and asks the sender to resend the message
- Coding
 - Creates a relationship between redundant bits and actual data bits; **block coding** and **convolution coding**

Modulo Arithmetic



- Modulo-2
 - Adding
 - $0+0=0$, $0+1=1$, $1+0=1$, $1+1=0$
 - Subtracting
 - $0-0=0$, $0-1=1$, $1-0=1$, $1-1=0$
 - Adding and subtracting yields the same results
 - We can use XOR for both addition and subtraction

$$0 \oplus 0 = 0 \qquad 1 \oplus 1 = 0$$

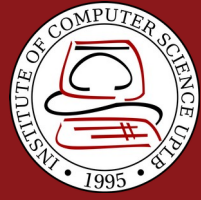
a. Two bits are the same, the result is 0.

$$0 \oplus 1 = 1 \qquad 1 \oplus 0 = 1$$

b. Two bits are different, the result is 1.

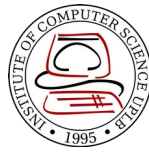
$$\begin{array}{rcccccc} & 1 & 0 & 1 & 1 & 0 \\ \oplus & 1 & 1 & 1 & 0 & 0 \\ \hline & 0 & 1 & 0 & 1 & 0 \end{array}$$

c. Result of XORing two patterns



Block Coding

Block Coding



- Divide message into **blocks**, each of **k bits**, called **datawords**
- We add **r** redundant bits to each block to make the length $n = k + r$, the n-bit blocks are called **codewords**
- With k bits, we can have 2^k **data words**, with n bits we can have 2^n **codewords**
- Same dataword is always encoded as the same codeword
- $2^n - 2^k$ codewords are not used (illegal/invalid)
 - Existence of these illegal codewords is the trick in error detection

4B/5B Block Coding



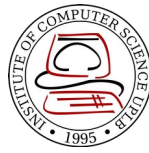
$k=4, n=5$

$2^k = 2^4 = 16$ datawords

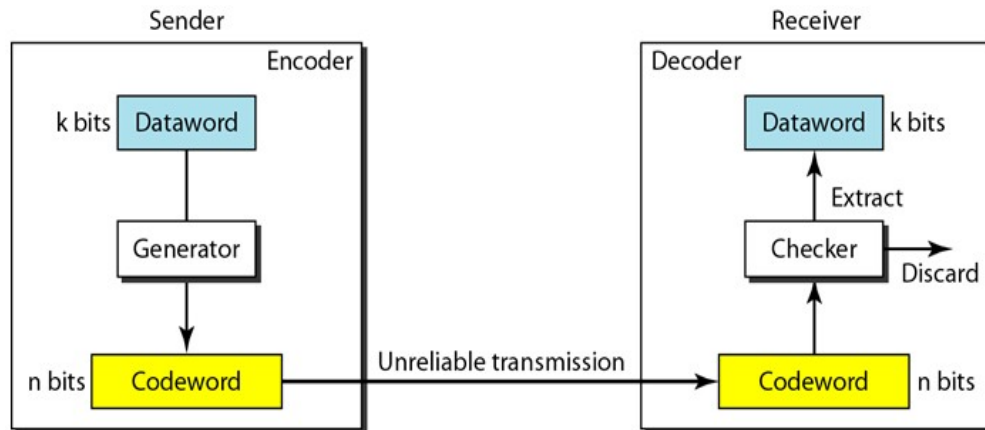
$2^n = 2^5 = 32$ codewords

<i>Data Sequence</i>	<i>Encoded Sequence</i>	<i>Control Sequence</i>	<i>Encoded Sequence</i>
0000	11110	Q (Quiet)	00000
0001	01001	I (Idle)	11111
0010	10100	H (Halt)	00100
0011	10101	J (Start delimiter)	11000
0100	01010	K (Start delimiter)	10001
0101	01011	T (End delimiter)	01101
0110	01110	S (Set)	11001
0111	01111	R (Reset)	00111
1000	10010		
1001	10011		
1010	10110		
1011	10111		
1100	11010		
1101	11011		
1110	11100		
1111	11101		

Block Coding: Error Detection



- Conditions to detect
 - Receiver has/can find a list of valid codewords
 - Original codeword has changed to an invalid one
- An error-detecting code can only detect the types of error for which it is designed; other types of error may remain undetected



Block Coding: Error Detection



Table A

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

*This can only detect single bit errors

- Example: Sender encodes 01 as 011
- Cases:
 - Receiver gets 011 - valid
 - Receiver gets 111 - invalid
 - Receiver gets 000 – undetected error

Block Coding: Error Correction

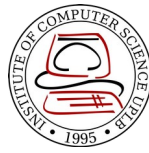


Table B

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

- Sender encodes 01 as 01011
- Receiver gets 01001
- Codeword not in table
- Assumes that 1 bit is corrupted
- Not first, third, fourth – 2 bits in error
- Must be the second!

Hamming Distance

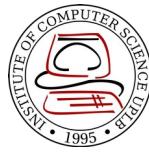


- The number of differences between the corresponding bits between two words

$$d(x,y)$$

- The Hamming Distance between the received codeword and the sent codeword is the number of bits that are corrupted during transmission
- Use **XOR** and count the **number of 1's** in the result
- Ex.: $d(000,011)=2$, $d(10101,11110)=3$

Minimum Hamming Distance

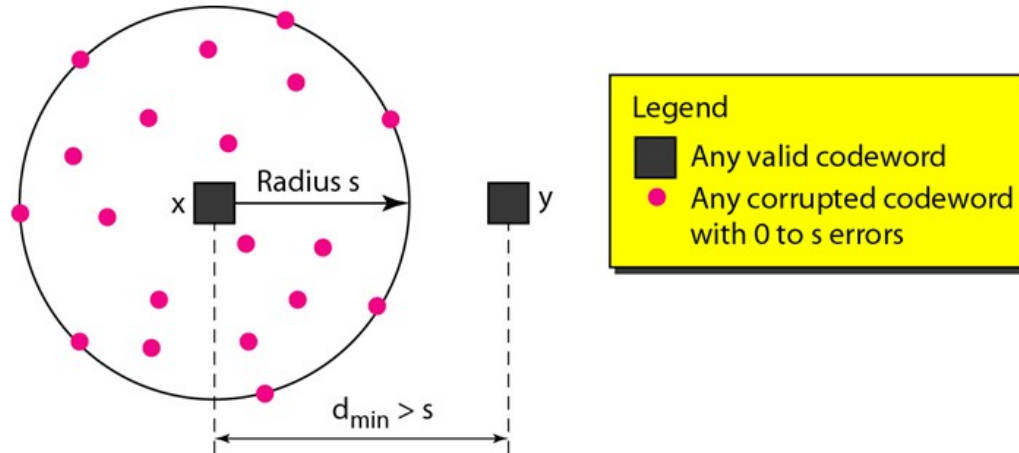


- Smallest hamming distance between all possible pairs of the codewords: $d_{\min}$
- From Table A: $C(3,2)$, $d_{\min}=2$
 $d(000,011)=2$, $d(000,101)=2$, $d(000,110)=2$,
 $d(011,101)=2$, $d(011,110)=2$, $d(101,110)=2$
- From Table B: $C(5,2)$, $d_{\min}=3$
 $d(00000,01011)=3$, $d(00000,10101)=3$, $d(00000,11110)=4$,
 $d(01011,10101)=4$, $d(01011,11110)=3$, $d(10101,11110)=3$
- Any coding scheme needs to have at least three parameters: **codeword size n** , **dataword size k** , and **$d_{\min}$**

Minimum Hamming Distance



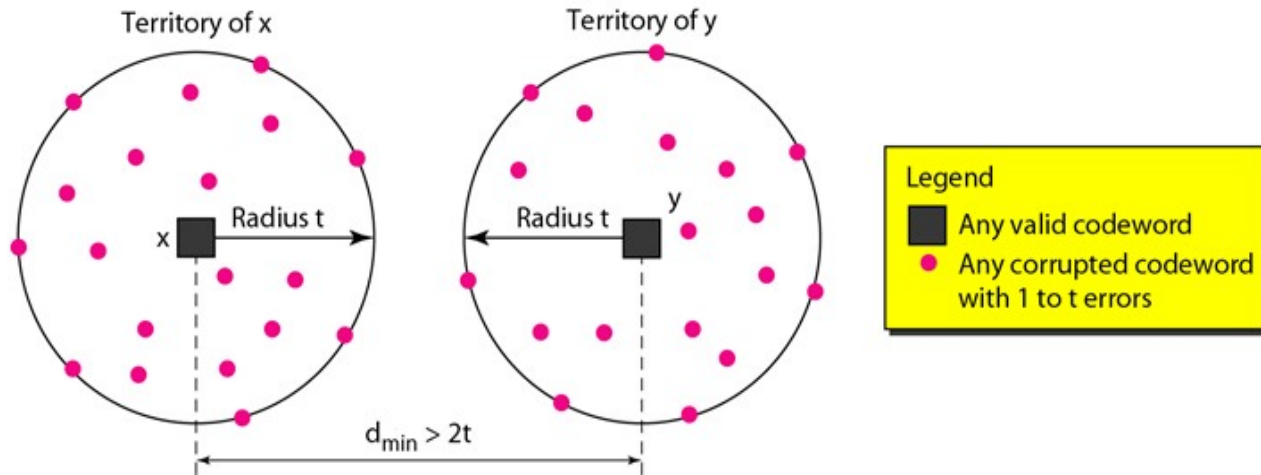
To guarantee **detection** of up to **s errors** in all cases, the minimum Hamming distance in a block code must be $d_{\min} = s + 1$



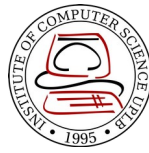
Minimum Hamming Distance



To guarantee **correction** of up to **t errors** in all cases, the minimum Hamming distance in a block code must be $d_{\min} = 2t + 1$



Linear Block Codes



- Almost all block codes used today belong to a subset of block codes called linear block codes
- A code in which the **XOR of two valid codewords** creates another **valid codeword**
- Minimum hamming distance is the number of 1's in the nonzero valid codeword with the smallest number of 1's
- In Table A, number of 1's in the nonzero codewords are 2 (011), 2 (101), and 2 (110); $d_{\min}=2$
- In Table B, number of 1's in the nonzero codewords are 3 (01011), 3 (10101), and 4 (11110) ; $d_{\min}=3$

Simple Parity Check



- A single-bit error-detecting code in which $n=k+1$ with $d_{\min}=2$
- Can only detect an odd number of errors

Simple Parity Check code C(5,4)

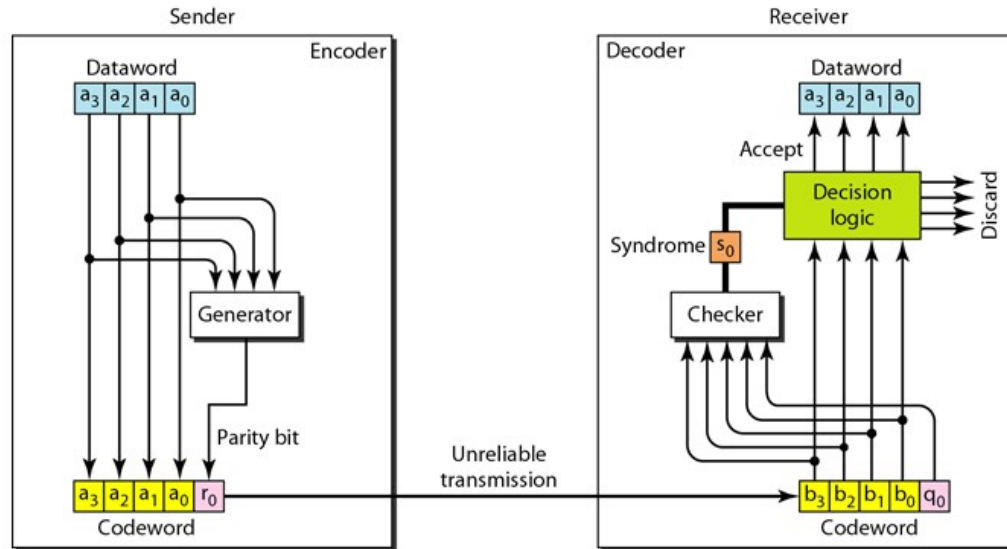
<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Simple Parity Check



$$r_0 = a_3 + a_2 + a_1 + a_0 \quad (\text{modulo-2})$$

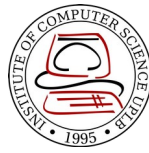
- number of 1's is even, result is 0; number of 1's is odd result is 1



$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \quad (\text{modulo-2})$$

- syndrome is 0 when the number of 1's in the received codeword is even; otherwise, it is 1

Cyclic Codes

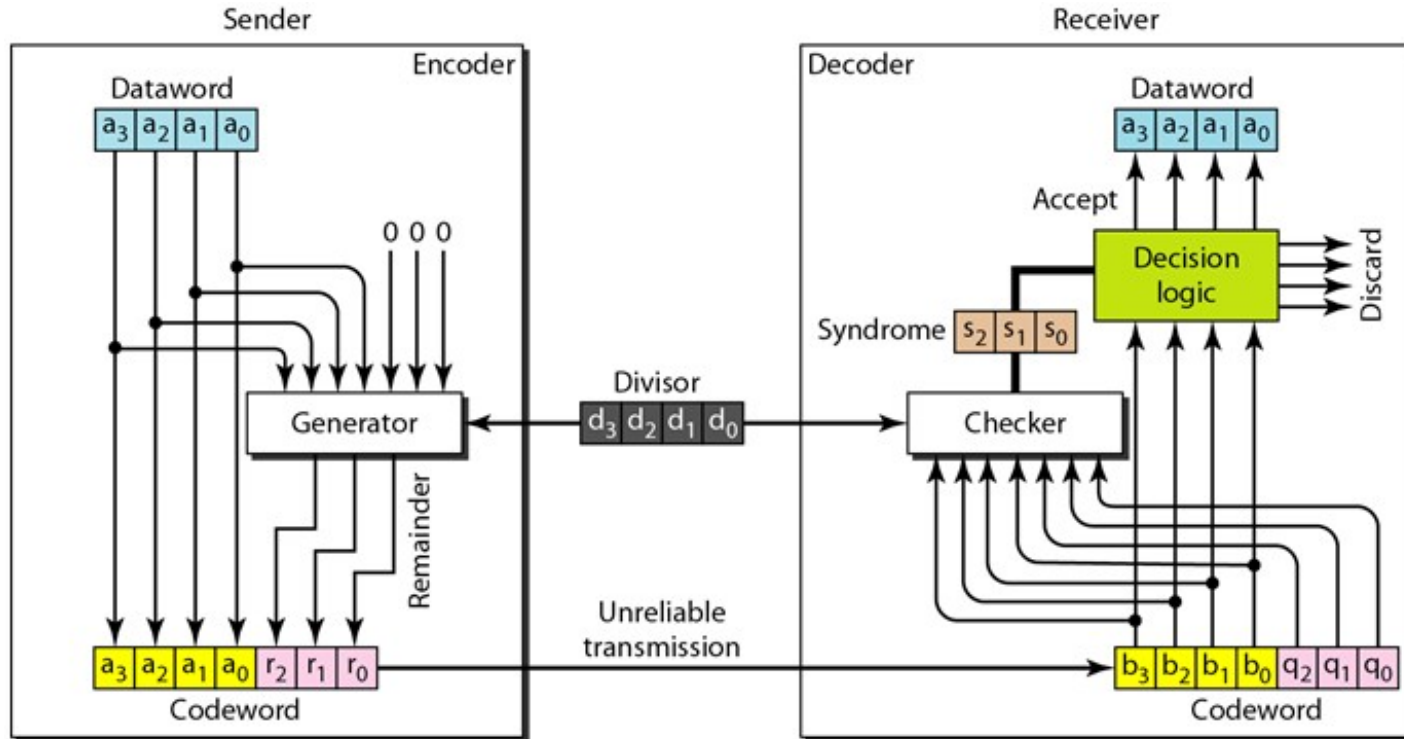


- Special linear block codes in which if a codeword is cyclically shifted (rotated), the result is another codeword
- Ex. 1011000, perform a SHL, 0110001
- Ex. Cyclic Redundancy Check (CRC)

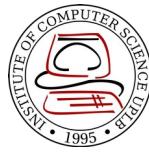
CRC Code with C(7,4)

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

CRC



CRC Encoder



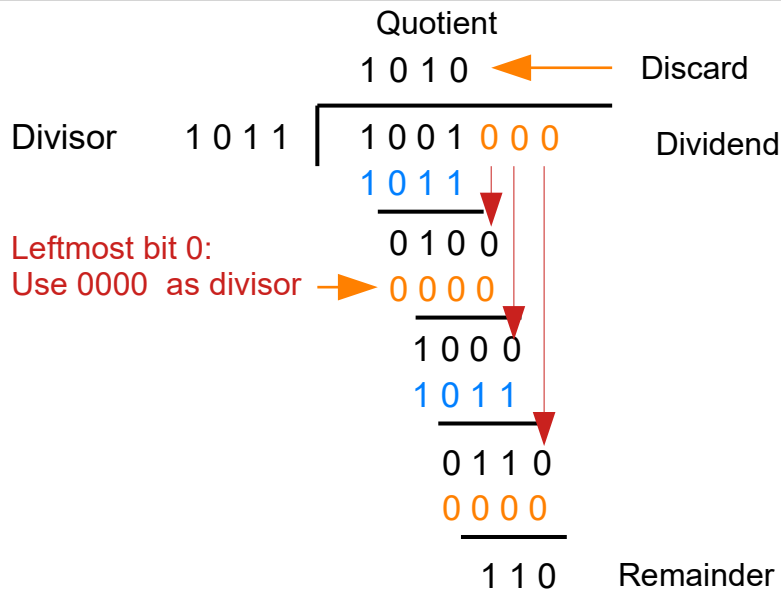
- Dataword has k bits, codewords has n bits
- Size of dataword is augmented by adding $n - k$ zeros to the righthand side
- n -bit result is fed into generator
- Generator use a divisor of size $n-k+1$, predefined and agreed upon
- Divisor divides augmented dataword by divisor (modulo-2 division)
- Remainder is appended to dataword to create codeword

CRC Encoder



Dataword

1001



Note:
Multiply: AND
Subtract: XOR

Encoder

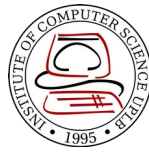
Codeword

1001 110

Dataword Remainder

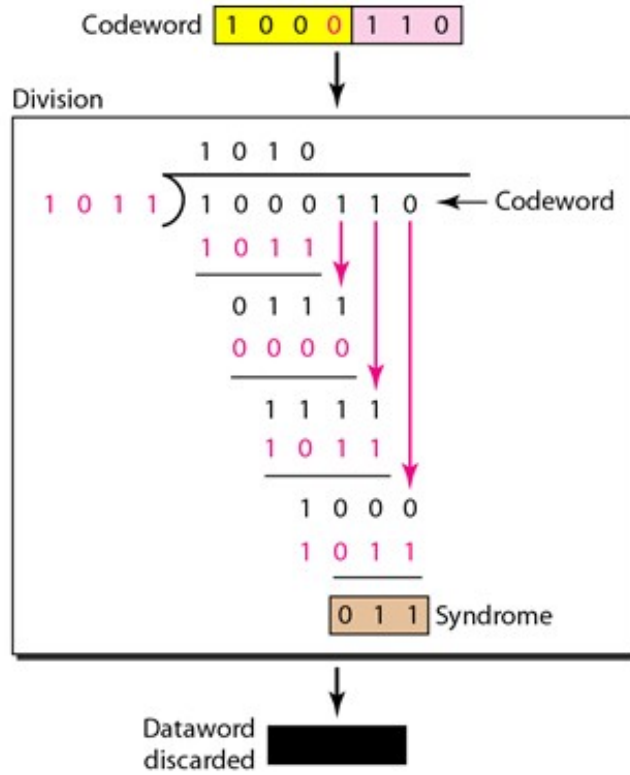
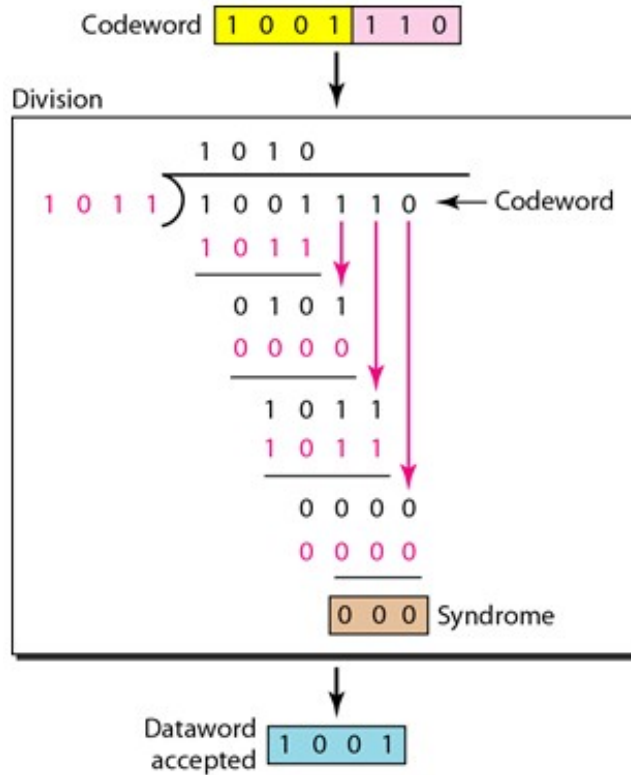


CRC Decoder

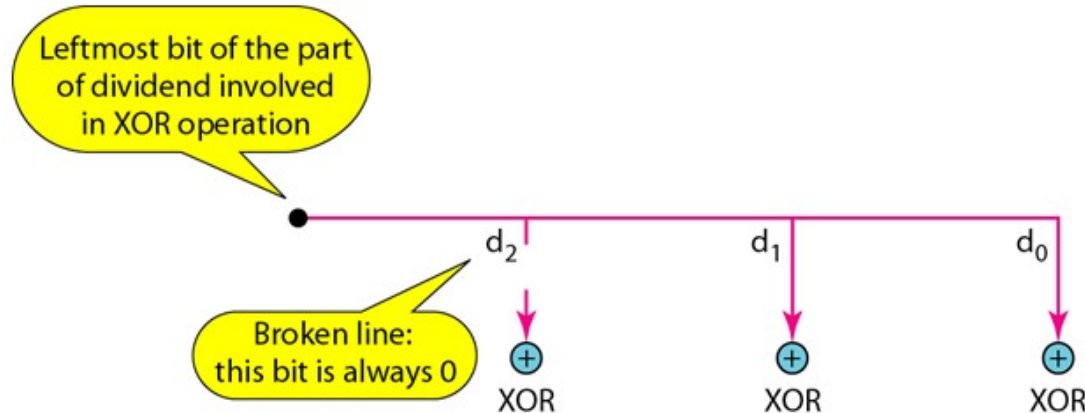


- Decoder receives a possibly corrupted codeword
- n bits fed to the checker, replica of generator
- Remainder produced by checker is a syndrome of $n-k$ bits
- If syndrome bits are all 0's, k leftmost bits of the codeword are accepted as dataword

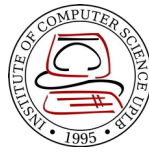
CRC Decoder



- Good performance in detecting single bit errors, double errors, odd number of errors, burst errors
- Easily implemented in hardware and software
- Hardware implementation is fast

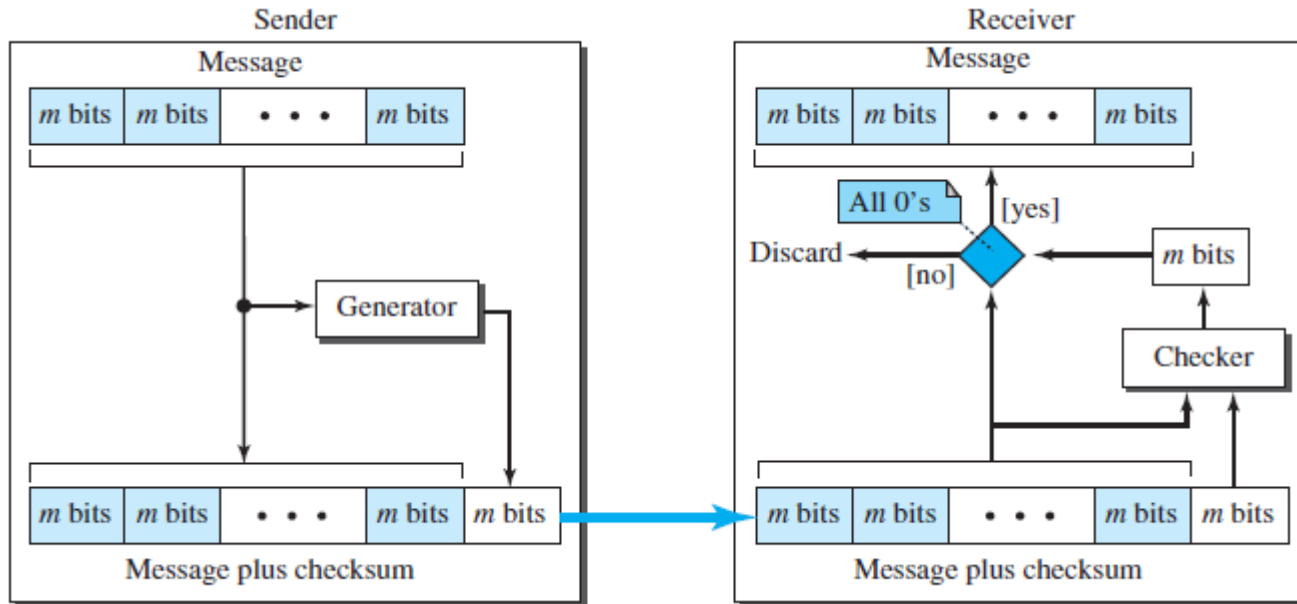
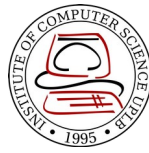


Checksum

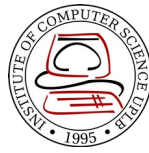


- Error-detecting technique that can be applied to a message of any length
- In the Internet, it is mostly used at the network and transport layer
- At the source, the message is first divided into m-bit units. The generator then creates an **extra m-bit unit** called the **checksum** which is sent with the message
- At the destination, the checker creates a new checksum from the combination of the message and the sent checksum. If the new checksum is all 0's, the message is accepted; otherwise it is discarded

Checksum



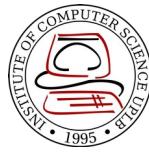
Checksum



Concept

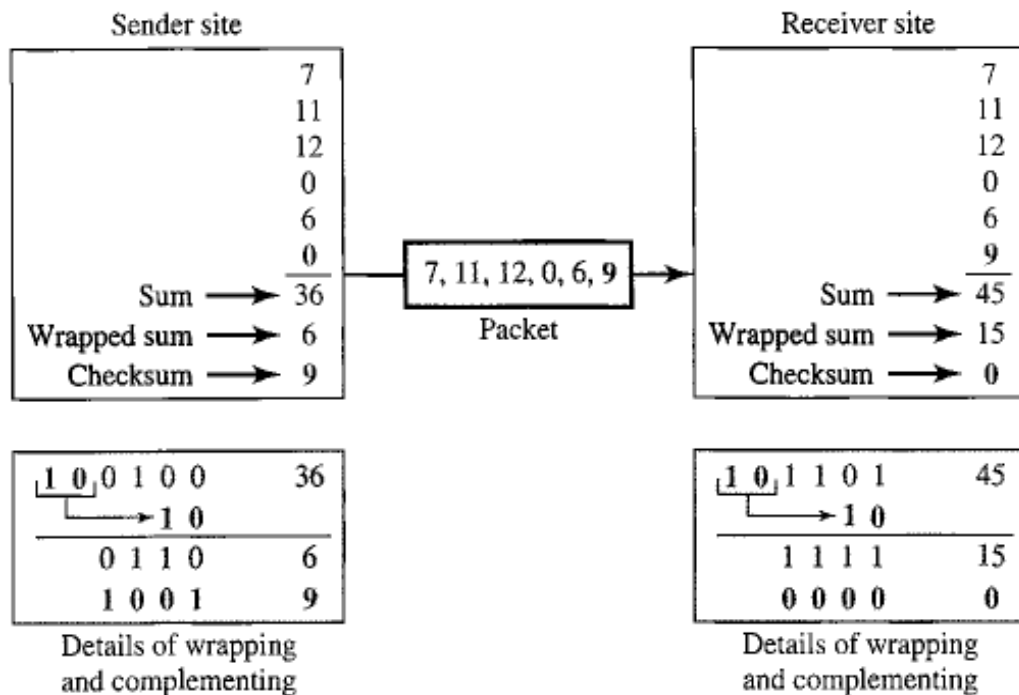
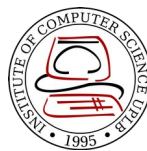
- Suppose data is five 4-bit numbers: (7,11,12,0,6)
- We send (7,11,12,0,6, 36), sum is 36
- Receiver adds the five number and compares result to the sum
- If we send the negative of the sum: (7,11,12,0,6, -36), receiver simply adds everything and a result of zero is expected

Checksum



- In the previous example, the checksum, -36, cannot be written as a four bit word
- Use one's complement arithmetic
 - Unsigned numbers between 0 to 2^n-1 using only n bits
 - If the number has more than n bits, extra leftmost bits are added to the n rightmost bits
 - Ex. 36 using 4 bits is 6;
 $100100 \longrightarrow 0100 + 10 = 0110$
 - Negative number can be represented by inverting all bits

Checksum



Checksum



Traditionally, the Internet has been using the 16-bit checksum

1	0	1	3	Carries
4	6	6	F	(Fo)
7	2	6	F	(ro)
7	5	7	A	(uz)
6	1	6	E	(an)
0	0	0	0	Checksum (initial)
8	F	C	6	Sum (partial)
8	F	C	7	Sum
7	0	3	8	Checksum (to send)

a. Checksum at the sender site

1	0	1	3	Carries
4	6	6	F	(Fo)
7	2	6	F	(ro)
7	5	7	A	(uz)
6	1	6	E	(an)
7	0	3	8	Checksum (received)
F	F	F	E	Sum (partial)
F	F	F	F	Sum
0	0	0	0	Checksum (new)

b. Checksum at the receiver site

References



- Forouzan, B.A. 2013. Data Communications and Networking, 5th Ed. McGraw-Hill, New York.